

W

Interpretable Control for Unstable Systems via SINDy-RL

Patrick Smith · Andrew Falcone

ME 595 · University of Washington · Spring 2026

What if the controller was just an equation?

$$u(x) = \underbrace{-2.4 \theta_1}_{\text{balance}} - \underbrace{0.9 \theta_2}_{\text{balance}} - \underbrace{0.5 \dot{\theta}_1}_{\text{damp}} - \underbrace{0.2 \dot{\theta}_2}_{\text{damp}} + \dots$$

8 terms · every coefficient has a physical interpretation · fits on a napkin

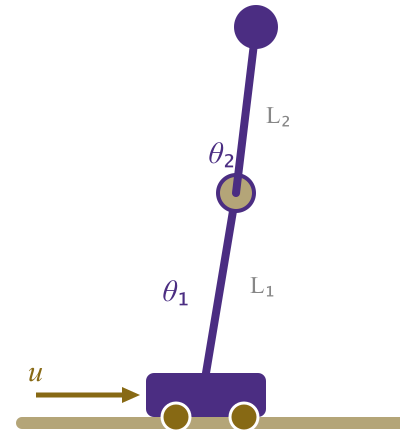
The Inverted Double Pendulum

State $x \in \mathbb{R}^6$

$$x = \begin{bmatrix} x_{\text{cart}} \\ \theta_1 \\ \theta_2 \\ \dot{x}_{\text{cart}} \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

Action $u \in [-1, 1]$ — cart force

Unstable equilibrium at $\theta_1 = \theta_2 = 0$



$L_1 = L_2 = 0.6 \text{ m}$ · $dt = 0.05 \text{ s}$ · MuJoCo physics

SINDy — Discovering equations from data

Key idea: dynamics live in a low-dimensional function space.

$$\dot{X} = \underbrace{\Theta(X, U)}_{\text{candidate feature library}} \cdot \underbrace{\Xi}_{\text{sparse coefficients}}$$

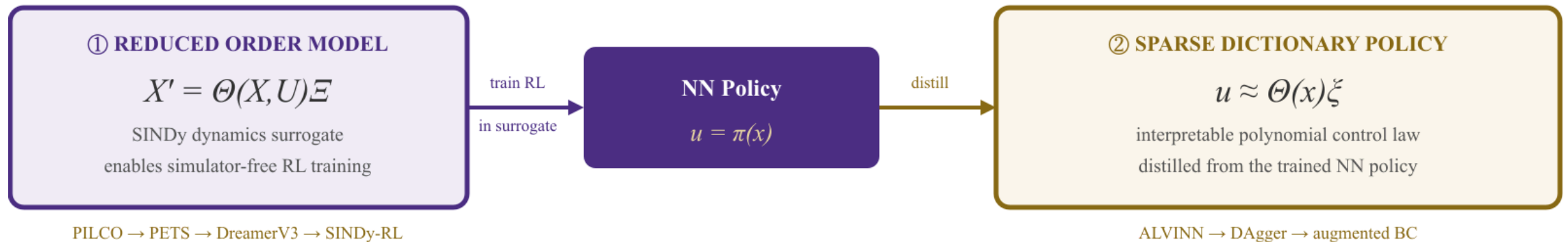
$$\Theta = \left[1 \mid x \mid \theta_1, \theta_2 \mid x^2, x\theta_1, \theta_1^2, \dots \right] \quad \text{degree-}d \text{ polynomial library}$$

STLSQ solver drives most coefficients to *exactly zero* — leaving a sparse equation with only the dominant governing terms.

Library degree d is a design choice — it must match the complexity of the system and is not obvious a priori.

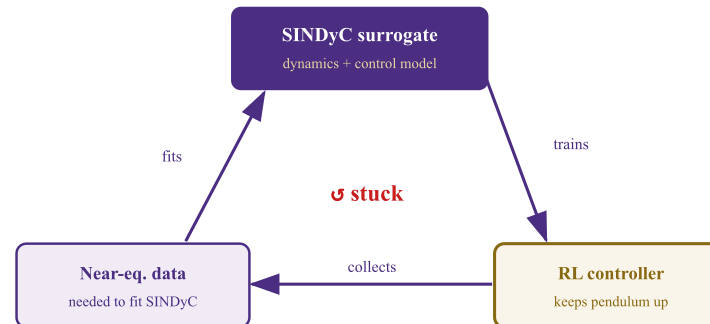
Lineage: LASSO '96 → SINDy '16 → SINDy-C '18 → E-SINDy '22 → **SINDy-RL '24** |
Koopman as the competing paradigm

Two objectives — one interpretable controller



SINDyC is a chicken-and-egg problem

You can't train near equilibrium without reaching it — and you can't reach it without a controller.



Solution: co-train the controller and surrogate in an iterative active-learning loop.

SINDy - open loop dynamics

SINDYc - co-training, bootstrap, and incrementally get better

Baseline — the performance ceiling

A standard PPO agent trained with **full simulator access**.

100%

Task success rate
(20/20 eval episodes)

9359

Mean reward
(max possible $\approx 10,000$)

9,731

Policy parameters
MLP [64 × 64]

50,000 transitions collected from the trained policy — the dataset used for sparse learning.

Sparse policy distillation — the compounding error trap

Behavioral cloning on the oracle dataset:

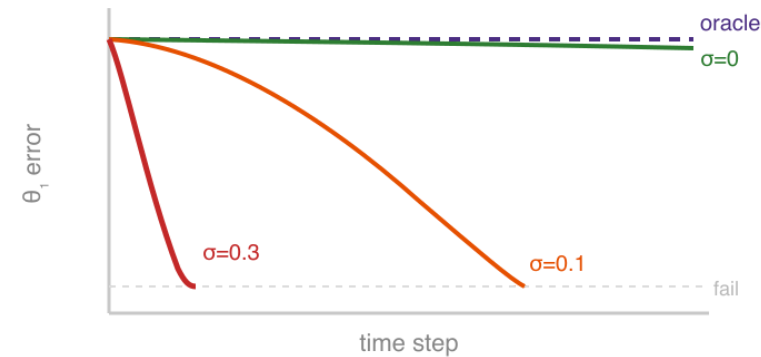
$$\min_{\Xi} \left\| \Theta(X) \Xi - U^* \right\|_2 + \lambda \|\Xi\|_1$$

Degree-4 library: 210 terms $\rightarrow$ STLSQ

selects **8 terms** ✓

But the policy fails in deployment.

At noise $\sigma = 0.3$: **1000 steps** $\rightarrow$ **~20 steps**



Noise σ	Mean episode length
0 (training)	~1000 steps ✓
0.1	~200 steps
0.3	~20 steps ✗

Off-distribution states produce small action errors $\rightarrow$ errors compound $\rightarrow$ catastrophic failure.

The fix: query the oracle, for free

Key insight: the NN policy is a pure function of state — no memory, no rollouts needed. Query it at *any* state we choose.

For each of 3 rounds:

1. Perturb states: $\tilde{x} = x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$
2. Query oracle: $\tilde{u} = \pi_{\text{NN}}(\tilde{x})$
3. Append $(\tilde{x}, \tilde{u})$ to dataset

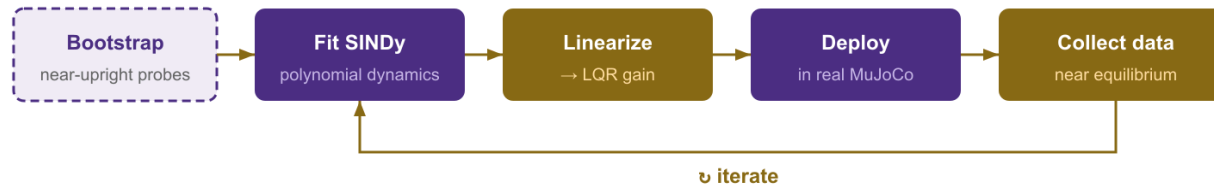
Dataset grows **4×** (50k $\rightarrow$ 200k pairs).
STLSQ re-fit recovers cross-coupling terms.

	$\sigma = 0.1$	$\sigma = 0.3$
Base policy	~200 steps	~20 steps ✗
Augmented	~1000 steps ✓	~500–900 steps

Baseline NN: 1000 steps at all noise levels

25–45× more robust — zero additional simulator interactions.

ROM surrogate — SINDy dynamics + LQR control



PPO Policy learned on SINDy fails in deployment

PPO found a policy that scored well *inside the polynomial* — by exploiting its approximation errors. Those actions don't generalize to the real system: **24** steps average in MuJoCo.

Why LQR transfers

LQR only uses the *Jacobian at the upright fixed point*. Near equilibrium, that linearization is accurate in both the polynomial model and the real system — so there are no model errors to exploit.

ROM surrogate — results

SINDy RMSE convergence (fixed validation set):

Iteration	RMSE	Cumulative transitions
0 (bootstrap)	0.188	5,000
1	0.182	10,000
2	0.085	15,000

Baseline NN: **400,000** real-sim steps.
SINDy-LQR: **15,000** — **27× more data-efficient.**

LQR transfer — real MuJoCo evaluation
(10 episodes per iteration):

Iteration	Mean return	Mean length	Success
0	9359.88	1,000	100%
1	9359.87	1,000	100%
2	9359.90	1,000	100%

LQR gain computed from SINDy linearized around upright.

Mean return matches the full-order baseline (9,359) exactly.

How do they compare?

Approach	Real-sim steps	Policy type	Mean length	Success	Interpretable	Data Efficiency
Baseline NN	400,000	NN (9,731 params)	1,000	100%	✗	-
Sparse policy (base)	400,000*	Polynomial (8 terms)	~20 @ $\sigma=0.3$	Low	✓	✓
Sparse policy (augmented)	400,000*	Polynomial (8 terms)	~500–900	~50–90%	✓	✓
Polynomial actor	1,000,000	Polynomial (22 terms)	1,000	100%	✓	✗
SINDy + PPO-in-surrogate	~15,000	PPO (NN)	~24	~0%	✗	✓
SINDy-LQR	15,000	LQR from SINDy	1,000	100%	✓	✓
Phase 3 (stretch)	TBD	Polynomial	—	—	✓	✓

* Inherits baseline training data, no additional agent training, one shot regression from

Stretch Goal

PHASE 3 · FULLY INTERPRETABLE CLOSED-LOOP CONTROL

Combine the interpretable **dynamics** (ROM) with the interpretable **policy** (sparse dictionary)

Bonus Stretch Goal

FOR FUN

2 chained PPO policies: swing-up
(energy pumping) + stabilizer.

NN baseline: Swing-up PPO →
handoff → Stabilizer PPO
304 steps · 15.2 s · **SUCCESS**

Goal: reproduce this with SINDy-RL —
interpretable swing-up + interpretable
stabilizer, end to end.



W

Interpretable control for unstable systems.

It works.

Patrick Smith · Andrew Falcone

ME 595 · University of Washington · Spring 2026